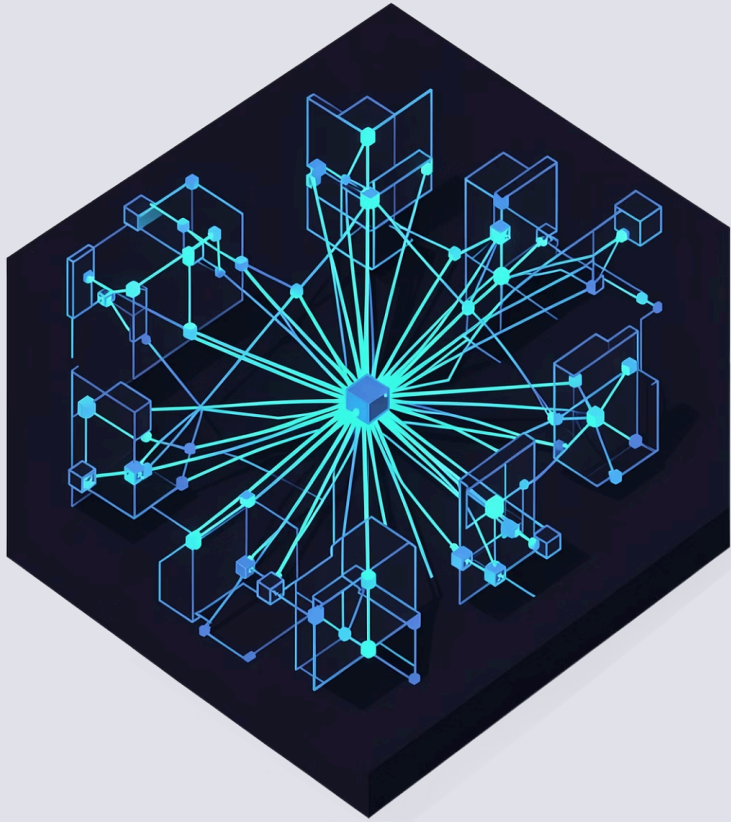




PyTorch Linear Regression Workflow

A beginner-friendly guide to building and training a linear regression model

Build a linear regression model using PyTorch

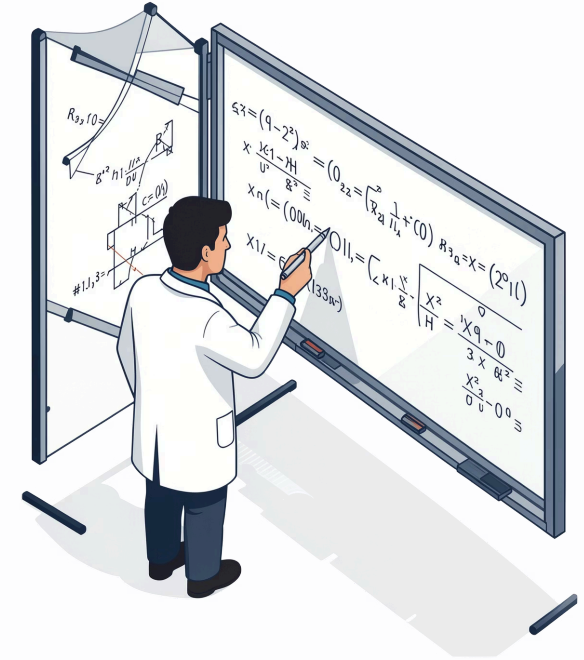
Learn the fundamental workflow of neural networks

$$y = \text{weight} \times x + \text{bias}$$

PyTorch 2.11.0

Data Generation

Synthetic data is created with **known parameters** so we can verify whether the model correctly learns the underlying relationship.



Weight

0.7

Bias

0.3

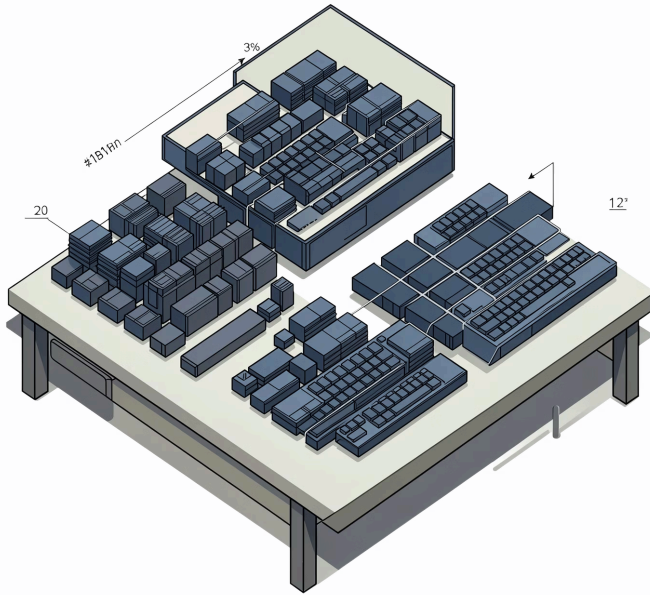
Formula

$$y = 0.7 \times X + 0.3$$

Data Range

X from 0.0 to 1.0 – 50 samples

Train / Test Split



Why Split the Data?

Splitting ensures the model is evaluated on data it has **never seen**, measuring how well it generalizes rather than memorizes.

This is the standard machine learning practice for preventing **overfitting** and validating model performance.

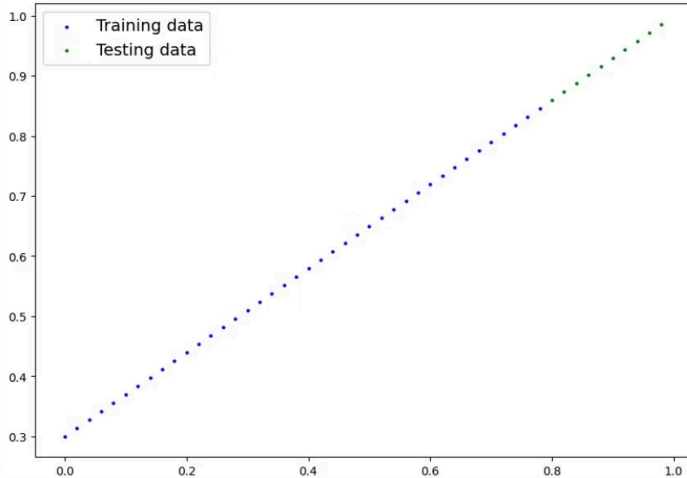
Training Set

40 samples — **80%**

Testing Set

10 samples — **20%**

Data Visualization



Reading the Chart

The scatter plot confirms a clean linear relationship across all 50 samples — exactly what we expect from our formula $y = 0.7x + 0.3$.

→ Blue dots

Training data — 40 samples used to teach the model

→ Green dots

Testing data — 10 samples held out for evaluation

→ Pattern

Clear linear trend with consistent slope and intercept

PyTorch Model Architecture



nn.Module Base Class

All PyTorch models inherit from `nn.Module`, giving them built-in parameter tracking and gradient support.



Learnable Parameters

Weight and bias are defined with `requires_grad=True` so PyTorch can compute gradients automatically.



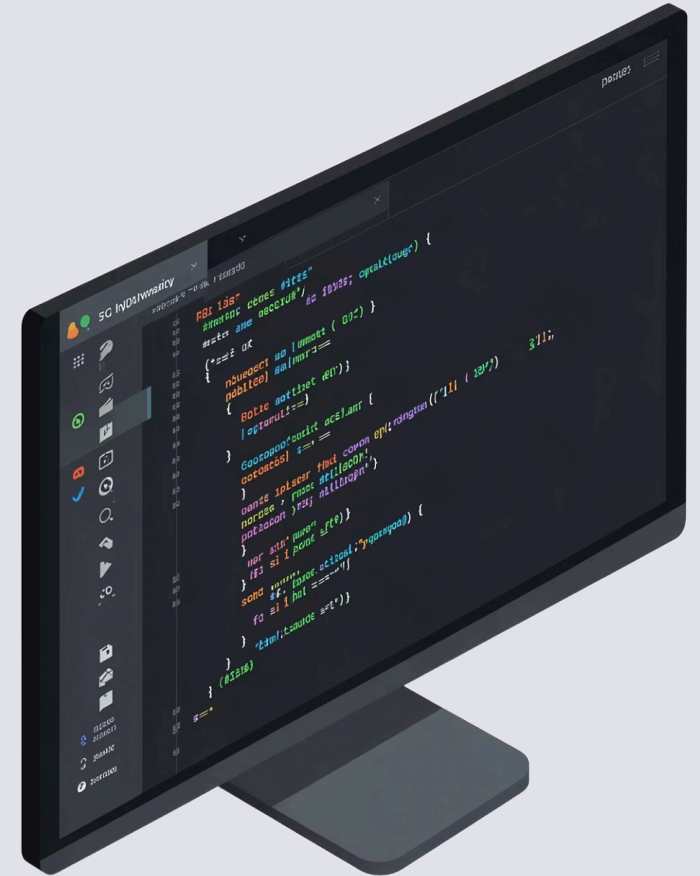
Forward Method

Implements $y = \text{weight} \times x + \text{bias}$ — the single computation performed on every input.

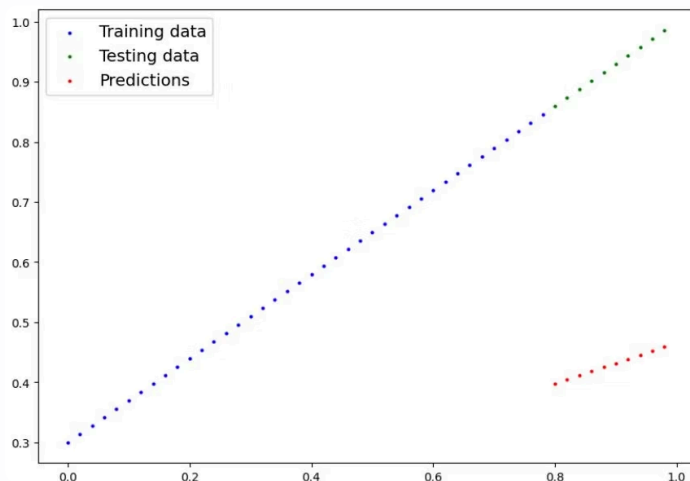


Reproducibility

Initial values are random but fixed with `torch.manual_seed` so results are consistent across runs.



Initial Predictions



Before Training

With randomly initialized parameters, the model's predictions are far off. This is the starting point the optimizer must correct.

The **red predictions** diverge significantly from the **green actual values** — training is needed to close this gap.

Random Weight

≈ 0.34

Random Bias

≈ 0.13

Loss Function & Optimizer



L1Loss

Mean Absolute Error — measures the average magnitude of difference between predictions and actual values



SGD Optimizer

Stochastic Gradient Descent — updates parameters by stepping in the direction that reduces the loss



Learning Rate: 0.01

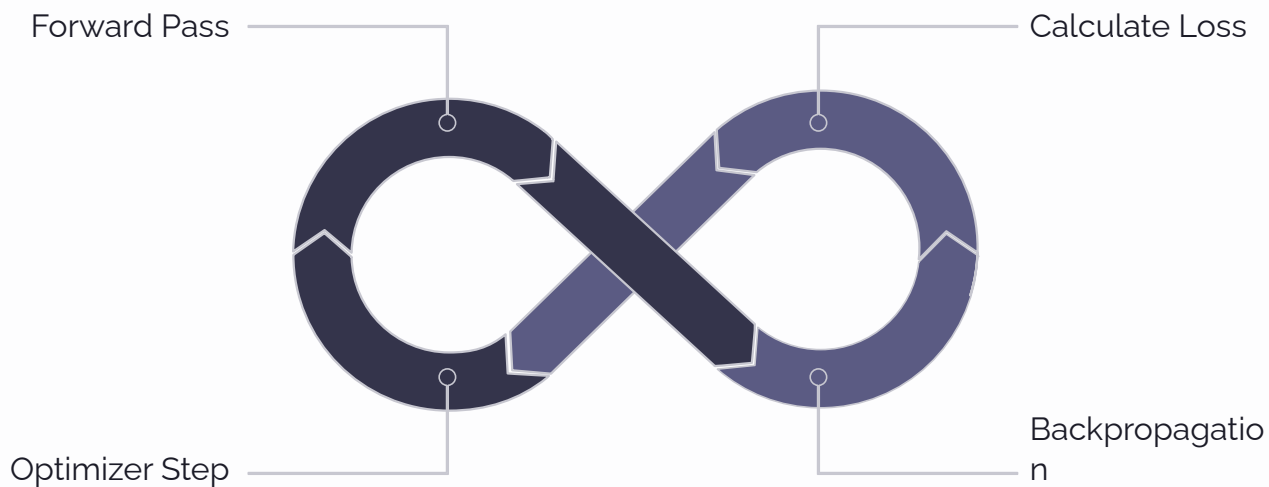
Controls the step size at each gradient update — small enough to converge, large enough to train efficiently



100 Epochs

The model passes through the full training dataset 100 times, refining parameters with each iteration

Training Loop

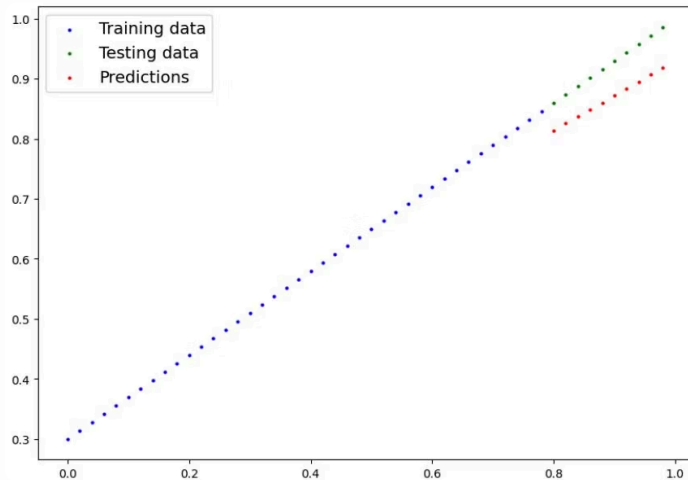


Loss decreases from **0.31** $\rightarrow$ **0.02** over 100 epochs — a clear sign the model is converging on the correct parameters.

Why Each Step Matters

- 1 Forward Pass**
Generates predictions from current weight and bias
- 2 Calculate Loss**
Quantifies how wrong the predictions are
- 3 Zero Gradients**
Prevents gradient accumulation from previous steps
- 4 Backpropagation**
Computes how each parameter contributed to the error
- 5 Optimizer Step**
Nudges weight and bias in the right direction

Final Predictions & Results



After 100 Epochs

The trained model's predictions now closely match the test data — confirming it has successfully learned the underlying linear relationship.

The **red predictions** now overlap tightly with the **green actual values** — the model has learned to generalize.

✅ Loss dropped from 0.31 to 0.02 — convergence confirmed.

Trained Weight

≈ **0.58** (target: 0.7)

Trained Bias

≈ **0.35** (target: 0.3)

Key Takeaways

01

Tensors & nn.Module

PyTorch's core data structure and base class for every neural network model

02

Learnable Parameters

Weights and biases with `requires_grad=True` enable automatic gradient computation

03

Forward Pass & Loss

Compute predictions, then quantify error with a loss function like L1Loss

04

Backpropagation & Optimizer

Gradients flow backward; SGD updates parameters to minimize loss each epoch

05

Training → Inference

Iterative forward → loss → backprop → update loop transforms a random model into a capable predictor